

Here is a classic scenario where a Snowflake query performs poorly due to common anti-patterns, followed by how to diagnose and fix it.

The Scenario: The E-Commerce Analytics Timeout

An analyst runs a daily report to find the **top 100 highest-spending customers** who bought items from a specific category ("Electronics") over the last **3 years**.

The query targets two tables:

1. `orders`: 5 billion rows (clustered naturally by `order_date`).
2. `order_items`: 20 billion rows.

The Poorly Performing Query

```
sql
SELECT
  c.customer_id,
  c.customer_name,
  SUM(oi.price * oi.quantity) AS total_spent
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN customers c ON o.customer_id = c.customer_id
WHERE UPPER(oi.category) = 'ELECTRONICS'
  AND DATE_TRUNC('year', o.order_date) >= '2023-01-01'
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spent DESC
LIMIT 100;
```

Why This Query Performs Poorly

If you look at the **Snowflake Query Profile**, you will see three massive performance bottlenecks:

1. Functions on `WHERE` Clause Columns (Destroys Pruning)

- **The Culprit:** `WHERE UPPER(oi.category) = 'ELECTRONICS' and DATE_TRUNC('year', o.order_date) >= '2023-01-01'`.
- **The Problem:** Applying functions to columns disables Snowflake's ability to perform **metadata pruning**. Snowflake has to scan 100% of the micro-partitions for both tables because it cannot compute the min/max values of a function on the fly.

2. Exploding Joins (Data Spilling)

- **The Culprit:** The `JOIN` happens before the data is aggressively filtered.
- **The Problem:** Snowflake has to load billions of rows into memory to perform the hash join. If the data exceeds the virtual warehouse's memory, it causes **Data Spilling to Remote Storage**. The query slows down significantly because it reads/writes to local disk or S3/Blob instead of memory.

3. High-Cardinality Group By

- **The Culprit:** `GROUP BY c.customer_id, c.customer_name.`
 - **The Problem:** Grouping by text fields (`customer_name`) alongside IDs forces Snowflake to pass large amounts of string data through the network during the aggregation phase.
-

The Tuned & Optimized Solution

To fix this, we rewrite the query to allow **partition pruning**, aggregate *before* joining text tables, and ensure data stays in memory.

The Optimized Query

```
sql
WITH filtered_items AS (
  -- Filter items directly using exact string match to allow pruning
  SELECT order_id, price, quantity
  FROM order_items
  WHERE category = 'Electronics'
),
filtered_orders AS (
  -- Filter orders using raw date logic to allow partition pruning
  SELECT order_id, customer_id
  FROM orders
  WHERE order_date >= '2023-01-01'
),
customer_totals AS (
  -- Aggregate by ID first to minimize data size before joining text
  SELECT
    o.customer_id,
    SUM(oi.price * oi.quantity) AS total_spent
  FROM filtered_orders o
  JOIN filtered_items oi ON o.order_id = oi.order_id
  GROUP BY o.customer_id
  ORDER BY total_spent DESC
  LIMIT 100
)
-- Join the small 100-row result set back to the Customer table for names
SELECT
  ct.customer_id,
  c.customer_name,
  ct.total_spent
FROM customer_totals ct
JOIN customers c ON ct.customer_id = c.customer_id
ORDER BY ct.total_spent DESC;
```

Summary of Performance Wins

Optimization	What Changed	Impact
Pruning enabled	Removed <code>UPPER()</code> and <code>DATE_TRUNC()</code> from <code>WHERE</code> clauses.	Scans 5% of the table instead of 100%.
Reduced Memory	Moved <code>customers</code> join to the very end.	Eliminates Spilling to Disk by keeping the join footprint tiny.
Faster Aggregation	Grouped by <code>customer_id</code> integer instead of strings.	Reduces network data shuffling across warehouse nodes.